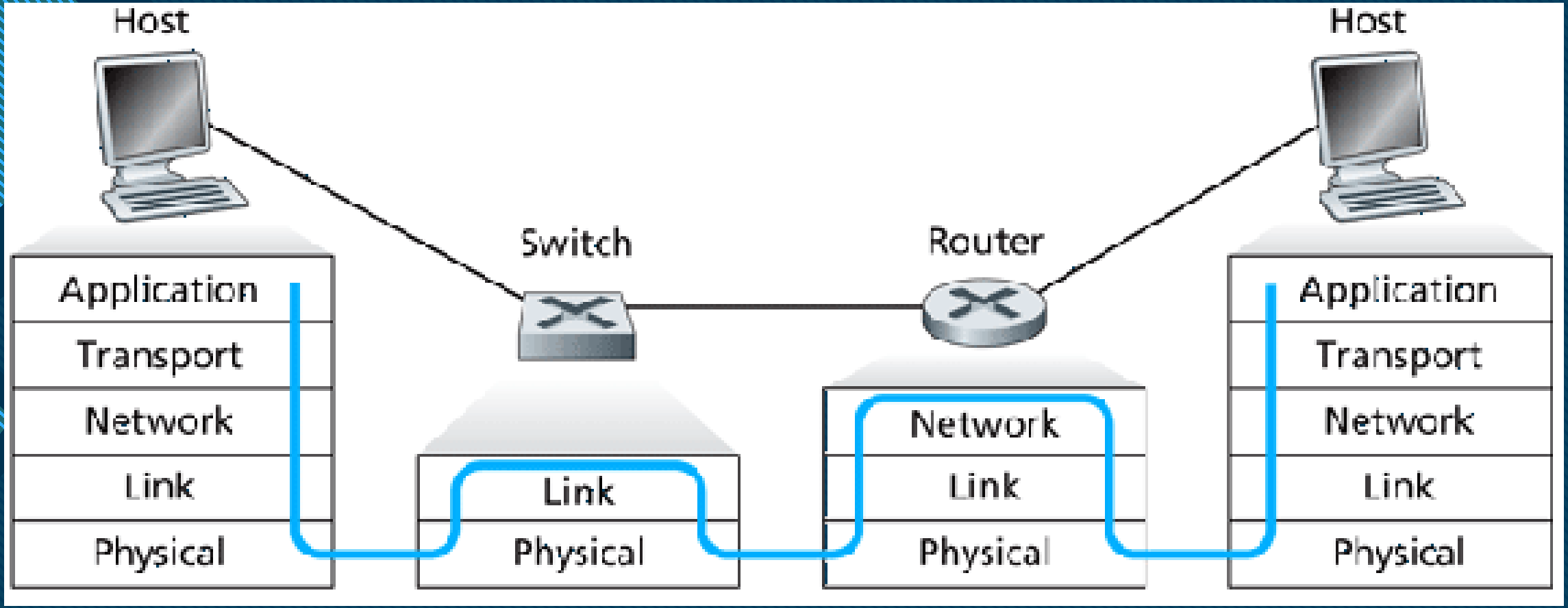
Computer Networks

Aftab Alam

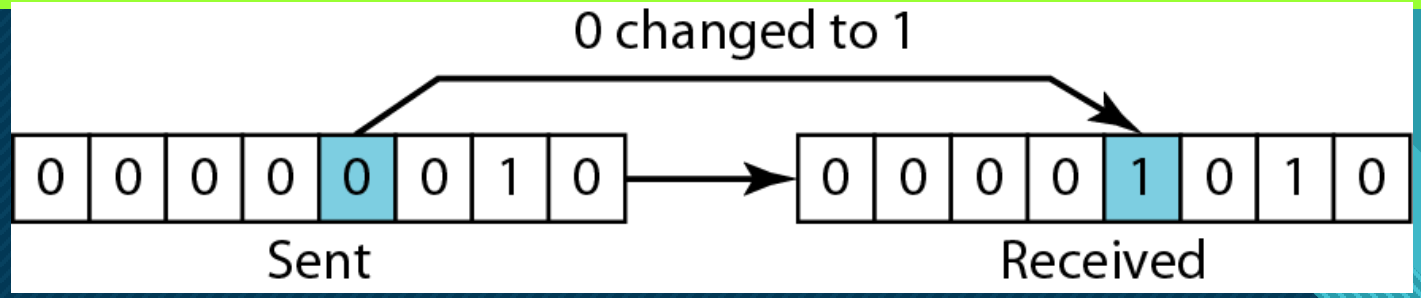
November 19, 2024 (Tuesday)

Lecture # 25

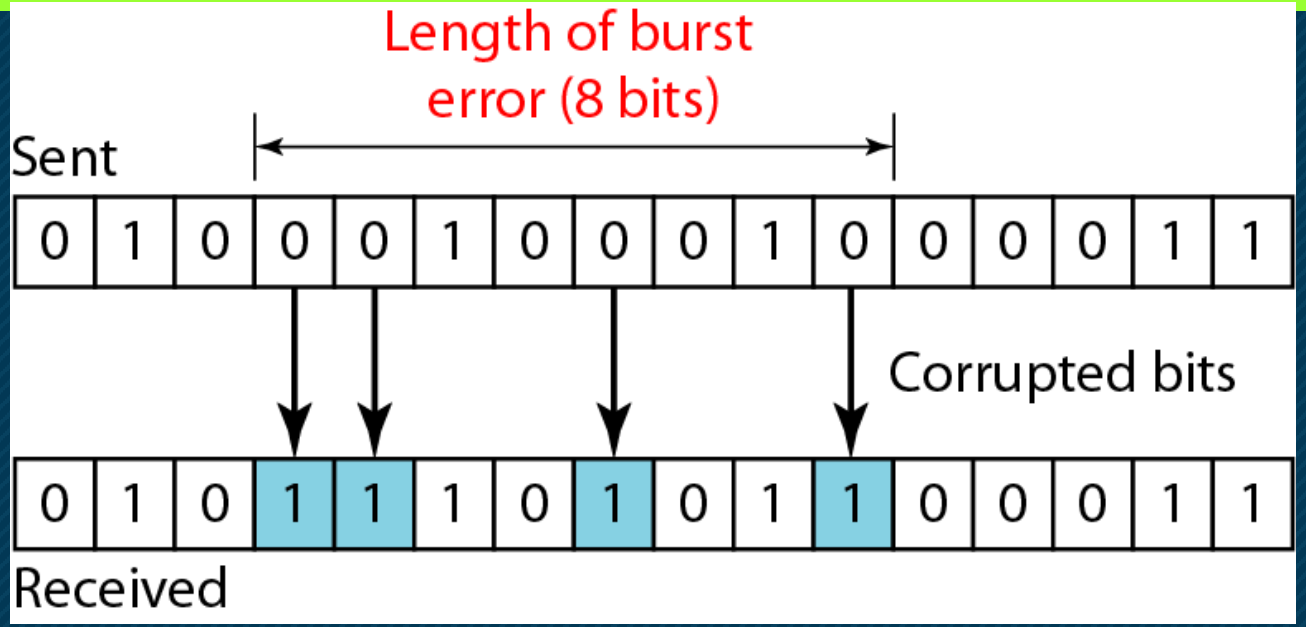
Link Layer



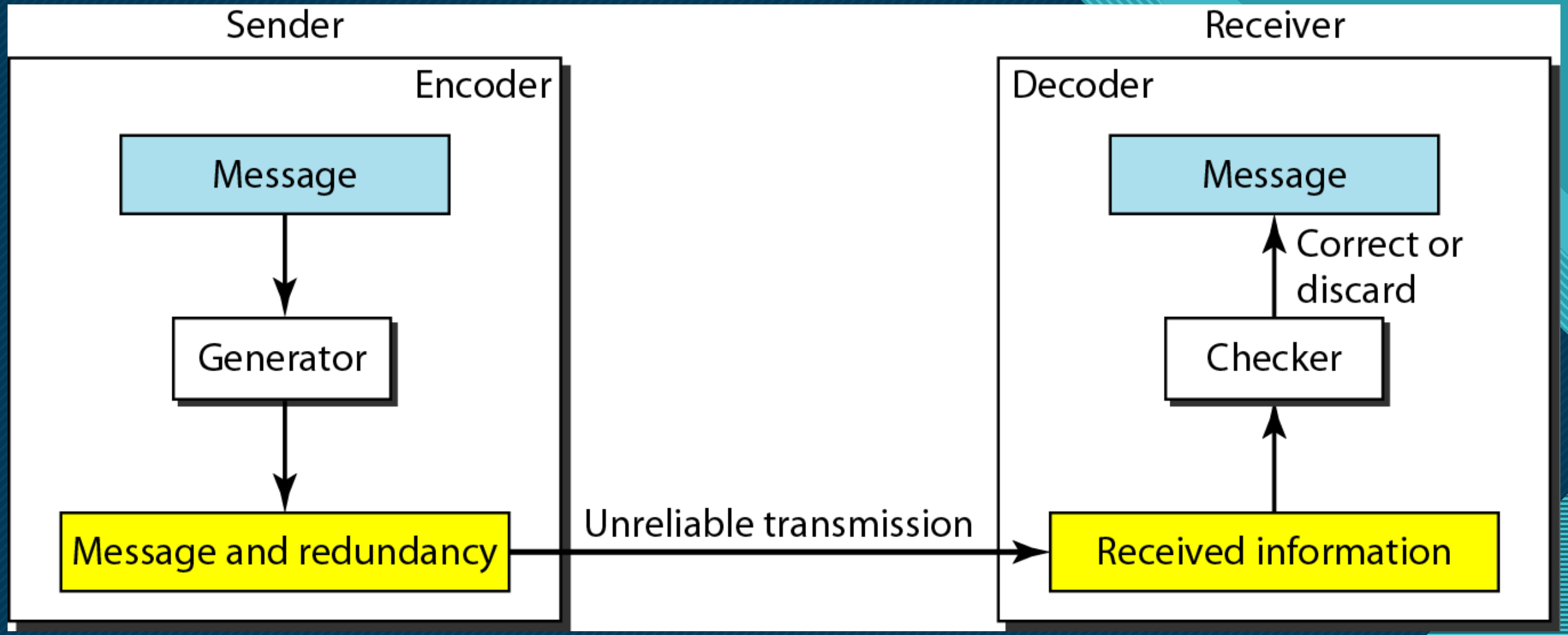
In a single-bit error, only 1 bit in the data unit has changed.



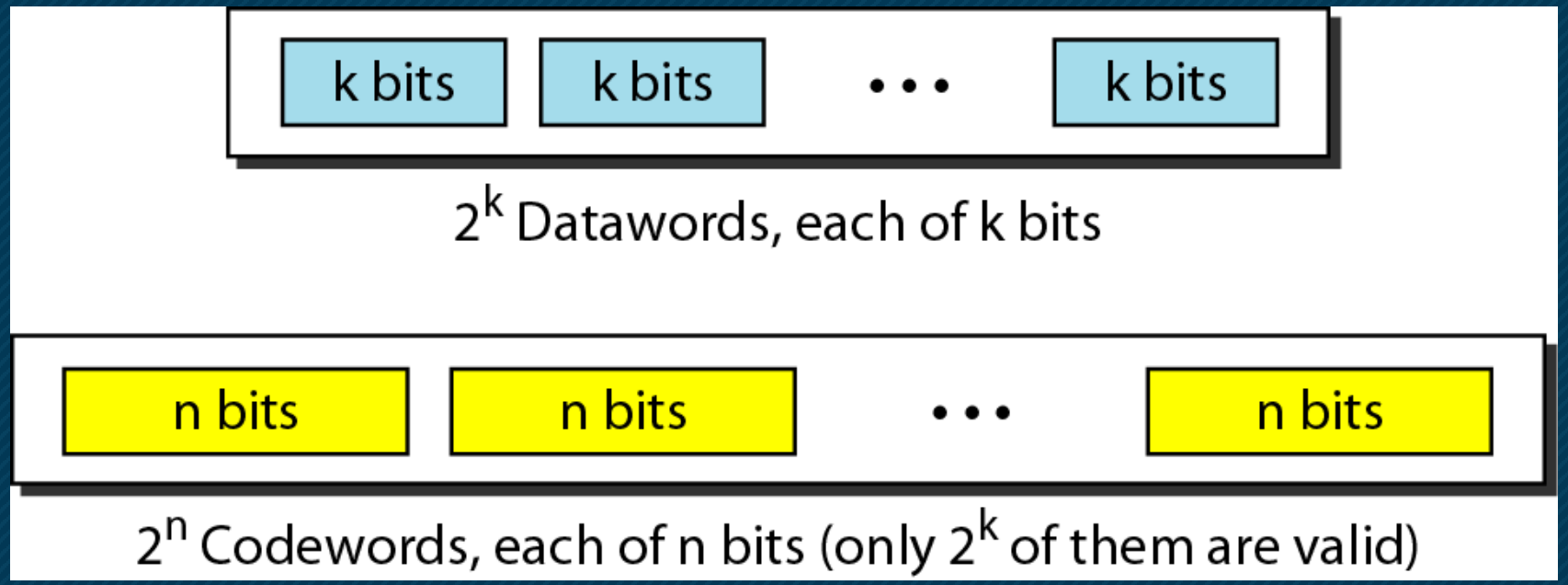
A burst error means that 2 or more bits in the data unit have changed.



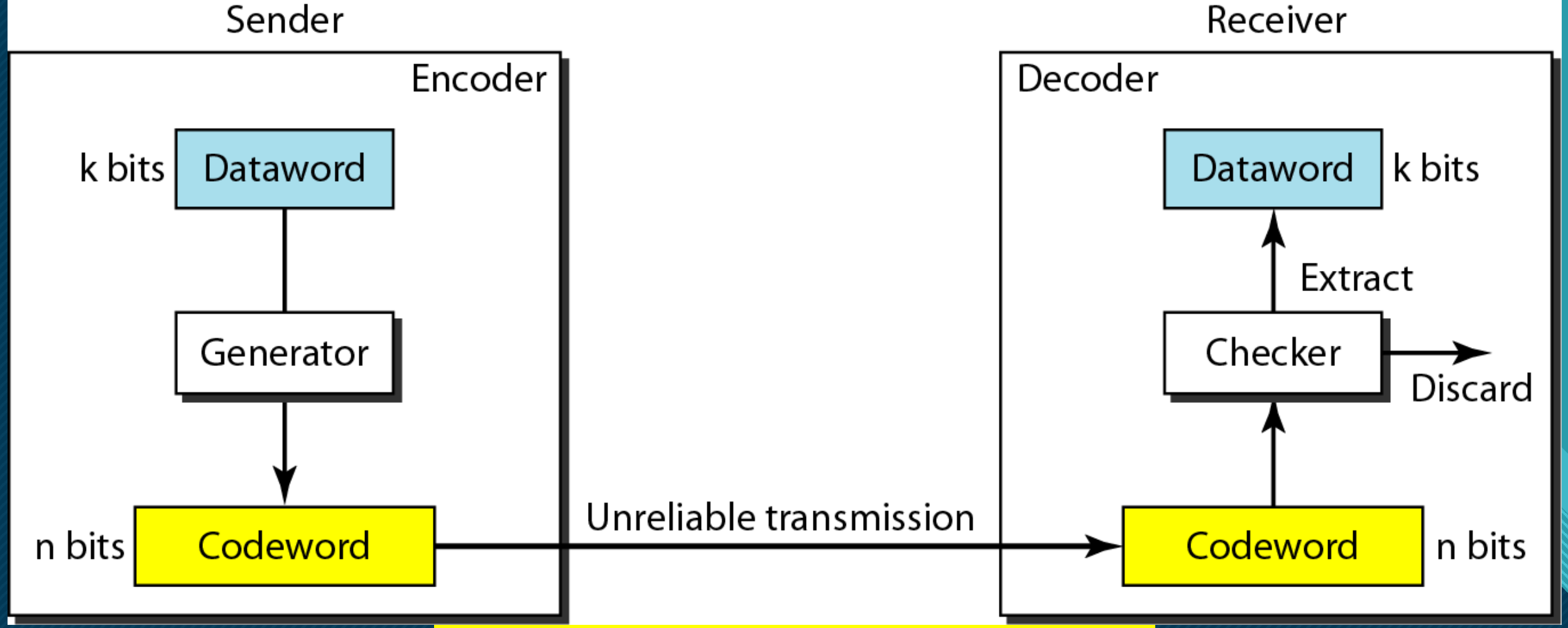
To detect or correct errors, we need to send extra (redundant) bits with data.



In block coding, we divide our message into blocks, each of k bits, called ***datawords****. We add r redundant bits to each block to make the length n = k + r. The resulting n-bit blocks are called* ***codewords****.*



Datawords and codewords in block coding



Process of error detection in block coding

*The 4B/5B block coding is a good example of this type of coding. In this coding scheme, $k = 4$ and $n = 5$. As we saw, we have $2^k = 16$ **datawords** and $2^n = 32$ **codewords**. We saw that 16 out of 32 codewords are used for message transfer and the rest are either used for other purposes or unused.*

Dataword : 01

Codeword: 011

1. *The receiver receives 011.*

   It is a valid codeword. The receiver extracts the dataword 01 from it.

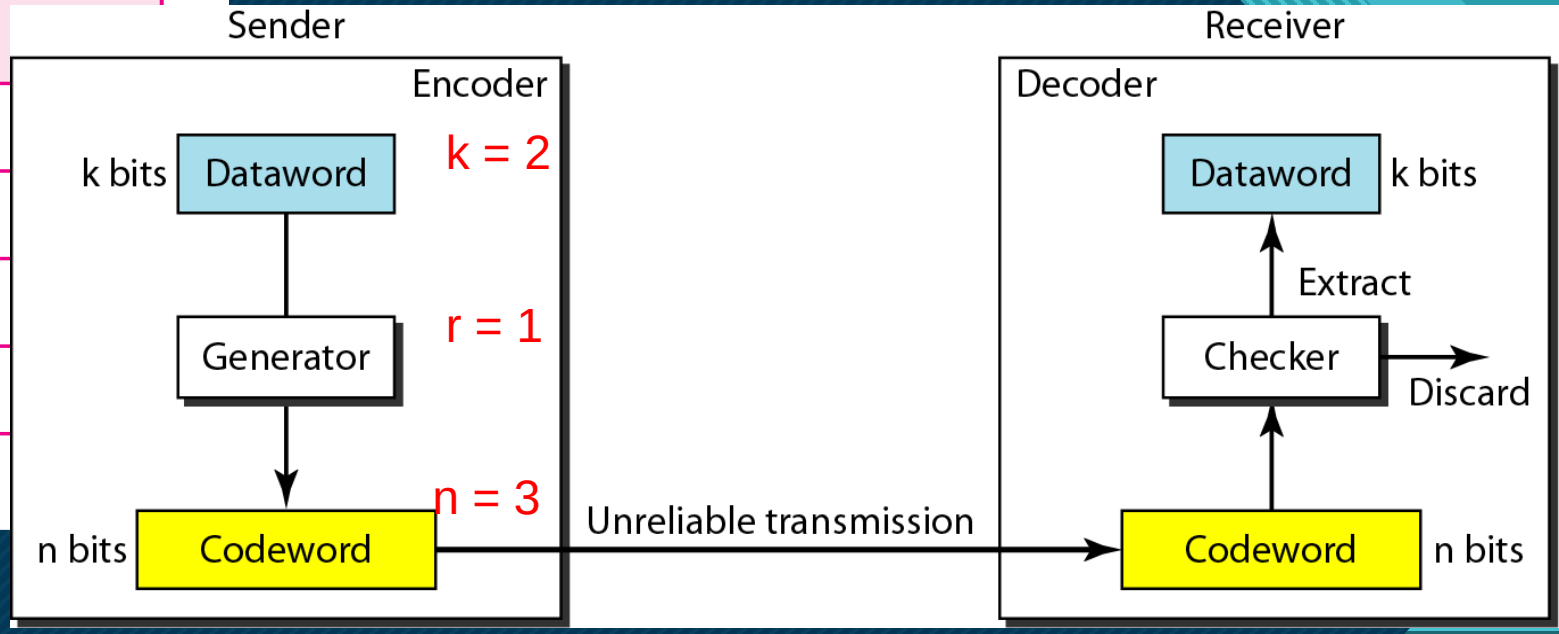
2. *The codeword is corrupted during transmission, and 111 is received.*

   This is not a valid codeword and is discarded.

3. *2-bits of the codeword is corrupted during transmission, and 000 is received.*

   This is a valid codeword. The receiver incorrectly extracts the dataword 00. Two corrupted bits have made the error undetectable.

Datawords	*Codewords*
00	000
01	011
10	101
11	110



An error-detecting code can detect only the types of errors for which it is designed; other types of errors may remain undetected.

Note

The Hamming distance between two words is the number of differences between corresponding bits.

The minimum Hamming distance is the smallest Hamming distance between all possible pairs in a set of words.

Datawords	*Codewords*
00	000
01	011
10	101
11	110

Hamming distances between all possible pairs of codewords:

d(000, 011) = 2
d(000, 101) = 2
d(000, 110) = 2
d(011, 101) = 2
d(011, 110) = 2
d(101, 110) = 2

Minimum Hamming distance: 2

To guarantee the detection of up to s errors in all cases, the minimum Hamming distance in a block code must be $d_{min} = s + 1$.

The minimum Hamming distance for our first code scheme (Error Detection Table) is 2. This code guarantees detection of only a single error. For example, if the third codeword (101) is sent and one error occurs, the received codeword does not match any valid codeword. If two errors occur, however, the received codeword may match a valid codeword and the errors are not detected.

Datawords	*Codewords*
00	000
01	011
10	101
11	110

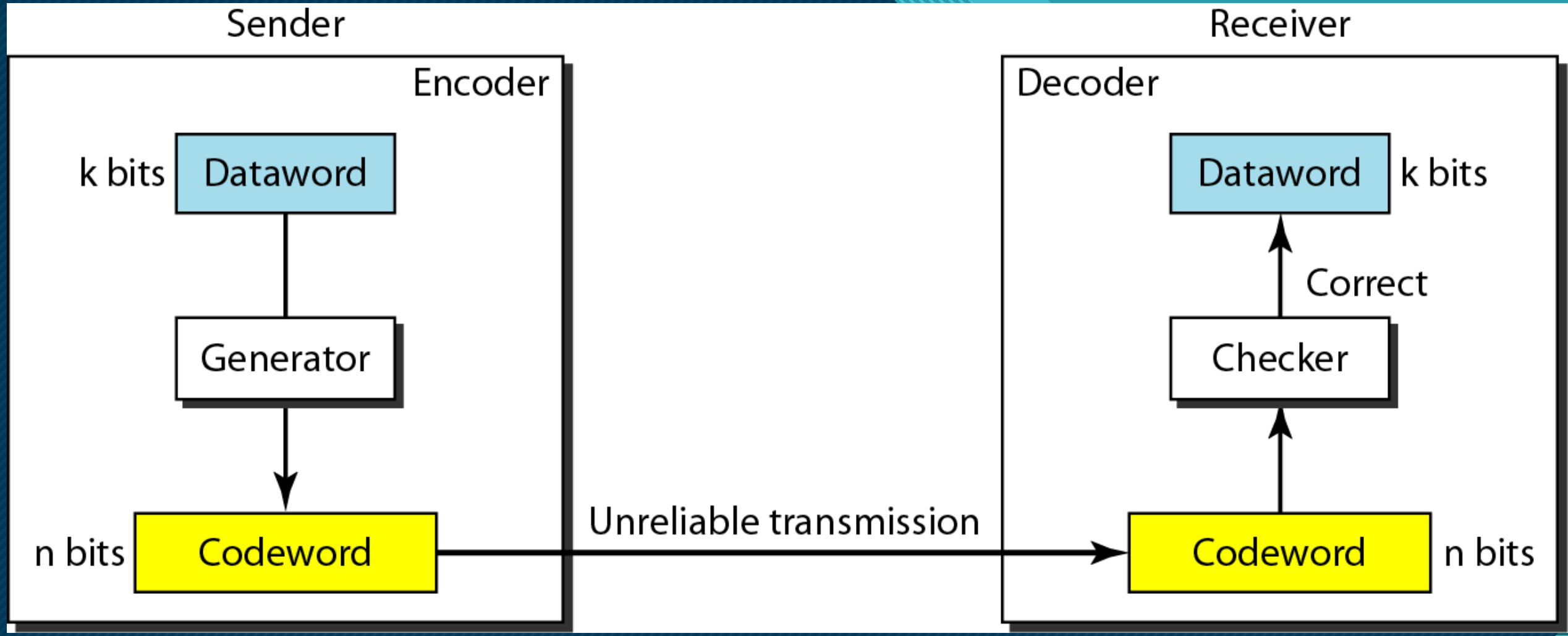
Dataword	*Codeword*
00	00000
01	01011
10	10101
11	11110

Hamming distances between all pairs of codes:

d(00000, 01011) = 3
d(00000, 10101) = 3
d(00000, 11110) = 4
d(01011, 10101) = 4
d(01011, 11110) = 3
d(10101, 11110) = 3

Minimum Hamming distance: 3

To guarantee correction of up to *t* errors in all cases, the minimum Hamming distance in a block code must be $d_{min} = 2t + 1$.



Structure of encoder and decoder in error correction

Let us add more redundant bits to previous Example to see if the receiver can correct an error without knowing what was actually sent. We add 3 redundant bits to the 2-bit dataword to make 5-bit codewords. Table shows the datawords and codewords.

Dataword	*Codeword*
00	00000
01	01011
10	10101
11	11110

*Assume the **dataword is 01**.*
*The sender creates the **codeword 01011**.*
*The codeword is corrupted during transmission, and **01001 is received**.*
*First, the receiver finds that the received codeword is **not in the table.***
This means an error has occurred. The receiver, assuming that there is only 1 bit corrupted, uses the following strategy to guess the correct dataword.

Find the Hamming Distance between received code 01001 and valid codes:

1. ***01001 XOR 00000 = 01001 (Hamming distance = 2)***
2. ***01001 XOR 01011 = 00010 (Hamming distance = 1)***
3. ***01001 XOR 10101 = 01001 (Hamming distance = 3)***
4. ***01001 XOR 11110 = 10111 (Hamming distance = 4)***

Our second block code scheme (Error Correction Table) has $d_{min} = 3$. This code can detect up to two errors. Again, we see that when any of the valid codewords is sent, two errors create a codeword which is not in the table of valid codewords. The receiver cannot be fooled.

However, some combinations of three errors change a valid codeword to another valid codeword. The receiver accepts the received codeword and the errors are undetected.

Dataword	*Codeword*
00	00000
01	01011
10	10101
11	11110

A code scheme has a Hamming distance $d_{min} = 4$. What is the error detection and correction capability of this scheme?

Solution

This code guarantees the detection of up to three errors ($s = 3$), but it can correct up to one error. In other words, if this code is used for error correction, part of its capability is wasted. Error correction codes need to have an odd minimum distance (3, 5, 7, . . .).

1. Given

$$\text{Message } D = 1010001101 \text{ (10 bits)}$$
$$\text{Pattern } P = 110101 \text{ (6 bits)}$$
$$\text{FCS } R = \text{ to be calculated (5 bits)}$$

Thus, $n = 15$, $k = 10$, and $(n - k) = 5$.

2. The message is multiplied by 2^5, yielding 101000110100000.

```
                        1 1 0 1 0 1 0 1 1 0  ← Q
P → 1 1 0 1 0 1 / 1 0 1 0 0 0 1 1 0 1 0 0 0 0 0  ← 2^n-
                  1 1 0 1 0 1
                    1 1 1 0 1 1
                    1 1 0 1 0 1
                        1 1 1 0 1 0
                        1 1 0 1 0 1
                            1 1 1 1 1 0
                            1 1 0 1 0 1
                                1 0 1 1 0 0
                                1 1 0 1 0 1
                                  1 1 0 0 1 0
                                  1 1 0 1 0 1
                                      0 1 1 1 0  ← R
```

4. The remainder is added to 2^5D to give $T = 101000110101110$, which is transmitted.

5. If there are no errors, the receiver receives T intact. The received frame is divided by P:

```
                        1 1 0 1 0 1 0 1 1 0  ← Q
P → 1 1 0 1 0 1 / 1 0 1 0 0 0 1 1 0 1 0 1 1 1 0  ← T
                  1 1 0 1 0 1
                    1 1 1 0 1 1
                    1 1 0 1 0 1
                        1 1 1 0 1 0
                        1 1 0 1 0 1
                            1 1 1 1 1 0
                            1 1 0 1 0 1
                                1 0 1 1 1 1
                                1 1 0 1 0 1
                                  1 1 0 1 0 1
                                  1 1 0 1 0 1
                                      0  ← R
```

$$
\begin{array}{rl}
 & X^9 + X^8 + X^6 + X^4 + X^2 + X \qquad \leftarrow Q(X) \\
P(X) \rightarrow X^5 + X^4 + X^2 + 1 \,\big/ & X^{14} \qquad X^{12} \qquad X^8 + X^7 + \quad X^5 \qquad \leftarrow X^5 D(X) \\
 & \underline{X^{14} + X^{13} + \quad X^{11} + \quad X^9} \\
 & \quad X^{13} + X^{12} + X^{11} + \quad X^9 + X^8 \\
 & \quad \underline{X^{13} + X^{12} + \quad X^{10} + \quad X^8} \\
 & \qquad X^{11} + X^{10} + X^9 + \quad X^7 \\
 & \qquad \underline{X^{11} + X^{10} + \quad X^8 + \quad X^6} \\
 & \qquad\quad X^9 + X^8 + X^7 + X^6 + X^5 \\
 & \qquad\quad \underline{X^9 + X^8 + \quad X^6 + \quad X^4} \\
 & \qquad\qquad X^7 + \quad X^5 + X^4 \\
 & \qquad\qquad \underline{X^7 + X^6 + \quad X^4 + \quad X^2} \\
 & \qquad\qquad\quad X^6 + X^5 + \quad X^2 \\
 & \qquad\qquad\quad \underline{X^6 + X^5 + \quad X^3 + \quad X} \\
 & \qquad\qquad\qquad X^3 + X^2 + X \leftarrow R(X)
\end{array}
$$

Four versions of $P(X)$ are widely used:

$$\text{CRC-12} = X^{12} + X^{11} + X^3 + X^2 + X + 1 = (X + 1)(X^{11} + X^2 + 1)$$

$$\text{CRC-ANSI} = X^{16} + X^{15} + X^2 + 1 = (X + 1)(X^{15} + X + 1)$$

$$\text{CRC-CCITT} = X^{16} + X^{12} + X^5 + 1 = (X + 1)(X^{15} + X^{14} + X^{13} + X^{12} + X^4 + X^3 + X^2 + X + 1)$$

$$\text{IEEE-802} = X^{32} + X^{26} + X^{23} + X^{22} + X^{16} + X^{12} + X^{11} + X^{10} + X^8 + X^7 + X^5 + X^4 + X^2 + X + 1$$

The CRC-12 system is used for transmission of streams of 6-bit characters and generates a 12-bit FCS. Both CRC-16 and CRC-CCITT are popular for 8-bit characters